

NN and CNNs for Image Classification/Recognition tasks

A) OBJECTIVES

1. Understand the implementation of a simple Deep NN implemented from scratch.
2. Learn how to implement and train a toy CNN with Keras for Image Classification
3. Fine-tune a well-known Image Classification CNN architecture with Keras

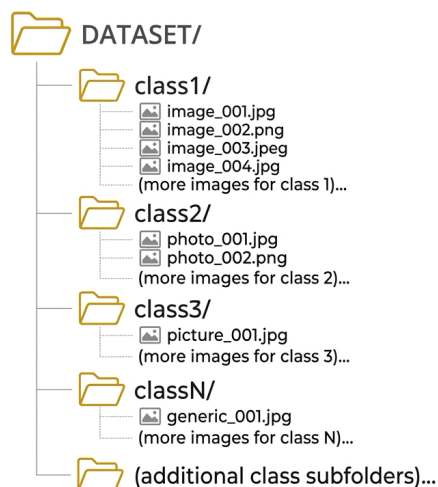
Optional task goals: variations of the DNNs and CNNs, segmentation, Pytorch

B) INTRODUCTION: In this practical session, we'll train and evaluate several deep neural networks for image classification. You'll use both the CIFAR-10 you've used in previous lab1 and your own tiny-dataset.

PREVIOUS TO THE SESSION: Select and download sample images for your own toy-5-class image classification problem. Pick 5 classes from some of these options (you can download just a few images, ~25-50, per class if you have constraints):

- <https://www.kaggle.com/kmader/food41/version/5#>
- <http://www.robots.ox.ac.uk/~vgg/data/pets/>
- <http://www.robots.ox.ac.uk/~vgg/data/flowers/>
- **yours?**

Create the dataset following the next format:



C) EVALUATION: Submit your notebooks to Moodle **by the corresponding deadline (Sunday, October 18th)**. To do this, just download the colab document as .ipynb and upload that to moodle. All your answers should be stored in the .ipynb. So please!! **MAKE SURE the notebook saves the outputs of your cells**. (If you didn't disable it, by default

it's done this way. You can download the notebook and make sure of it before closing your session).

The following will be evaluated:

- Correctness and extent of the work. (NO NEED for HIGH accuracies! These are toy-examples. We just want correctness on what you have done!)
- Correctness in the use of technical terms in the answers.
- Analysis and discussion of the results.
- Organization and cleanliness of the code.

REMEMBER:

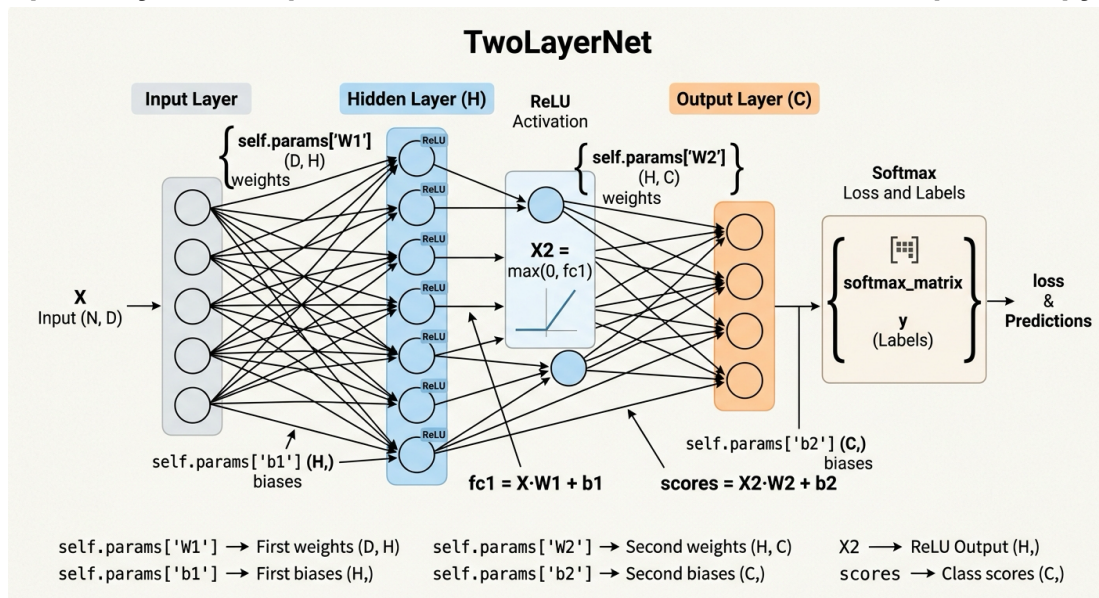
- To opt for the maximum grade, **REQUIRED** to submit all mandatory tasks + the optional task,

- Work in a groups of two as in the previous labs.

D) DESCRIPTION OF MANDATORY TASKS for LAB3:

For each task, answer ALL the questions within the notebook, add the code to fill the gaps and run the requested subtasks:

1. Simple 2 layer NN implemented from scratch. Use notebook **Simple2NN.ipynb**.



- 1.1 Understand and complete the code
- 1.2 Run it to train the model with toy data
- 1.3 Repeat with CIFAR-10
- 1.4 Answer the questions.

2. Simple CNN implemented in Keras. Use notebook **SimpleCNN.ipynb**.

- 2.1 Understand the example and run it to train your own toy-dataset.

- 2.2 Define a new model (model2). What happens if you remove the dropout layers?
- 2.3 Add a new model (model3) including data augmentation.

Re-train the CNN on your toy-dataset to analyze what these changes do and briefly discuss the results of the CNN with/without augmentation, with/without dropout.

3. Fine-tune existing CNN with Keras. Use notebook **CNN_Finetunning.ipynb**.

- 3.1. Fine-tune an existing well-known CNN architecture for Image classification on your toy-dataset.
- 3.2. Compare and discuss the results you obtain for your toy-dataset on this task and in previous one/s.

E) Optional task:

- **Semantic segmentation.** Use the OPT_TorchVision_SemantSeg.ipynb notebook 4.1. (1 POINT) Read, modify and analyze the results of different semantic segmentation models.